Branches & Style Guides

Week 2 - 06/22/2026

LEARNING GOALS:

Learn about the causes of merge conflicts
Practice making branches and switching between them
Resolve merge conflicts using GitHub's interface

What are Merge Conflicts?

What are Merge Conflicts?

An incompatible change between two versions of a file

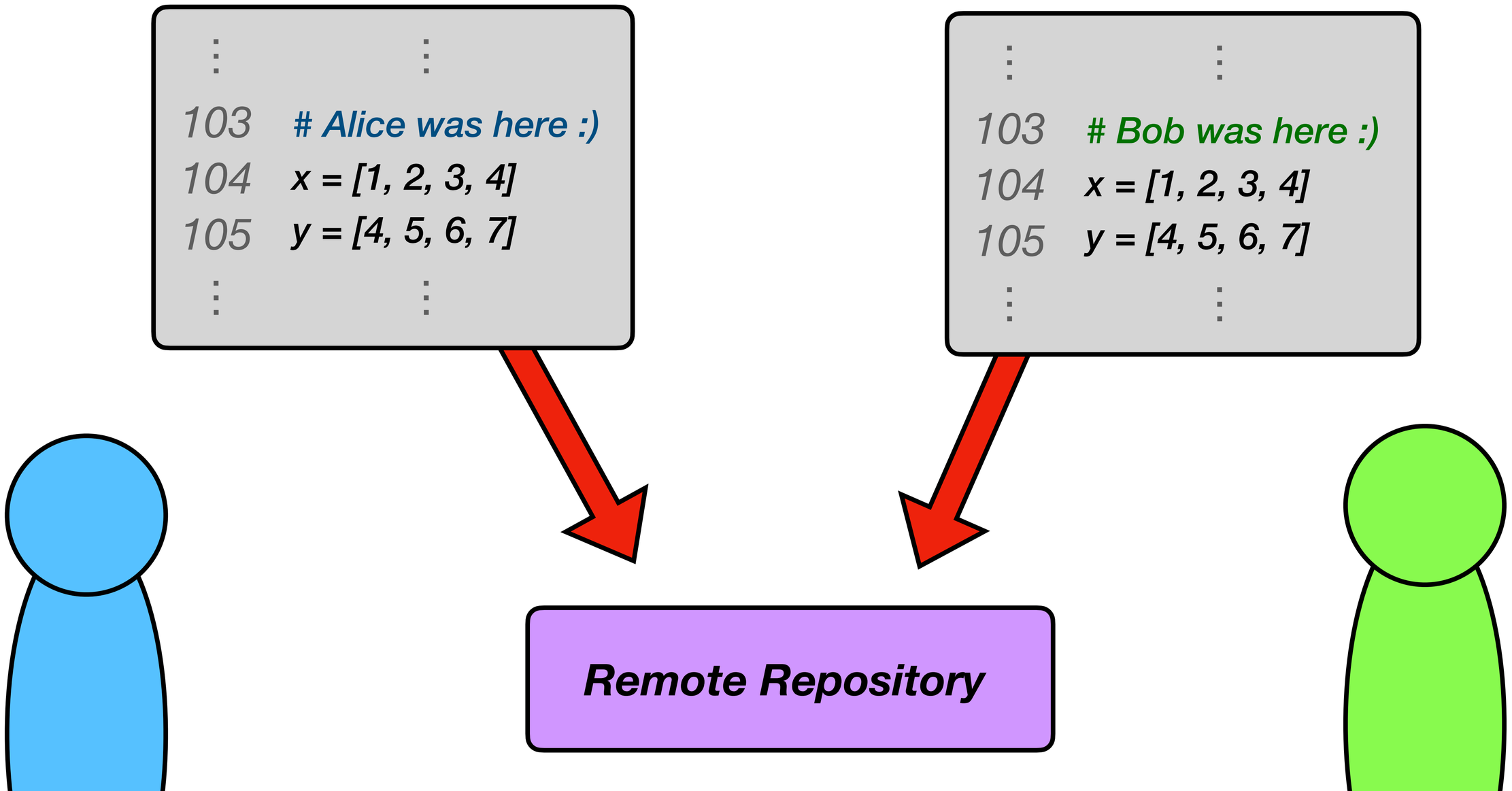
An update to a file that has been deleted

An update to a file that has been renamed

⋮

Any situation where Git cannot figure out which change to keep!

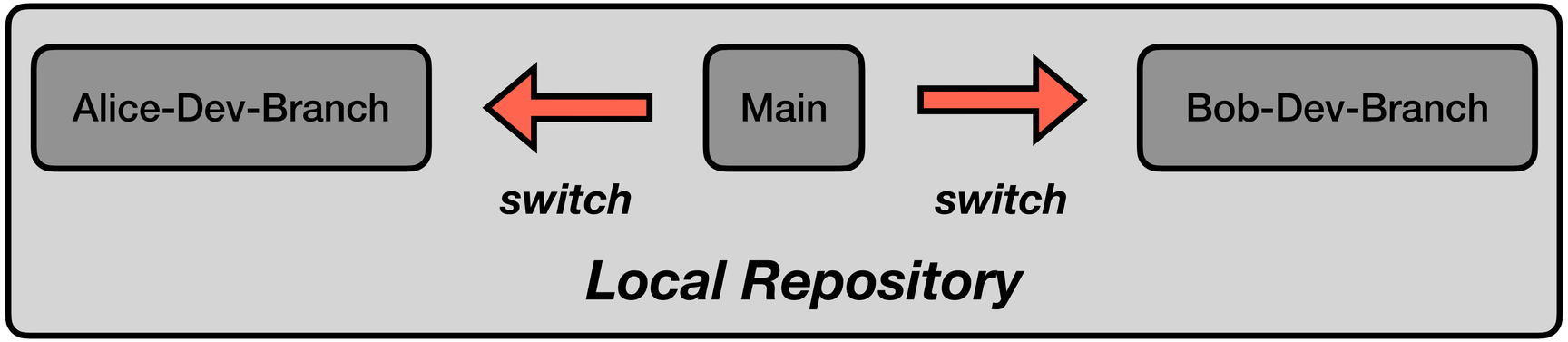
If two coders change the same line of code, which one should be kept?



Branches Make Things Easier!

Each branch is a “separate local repository”

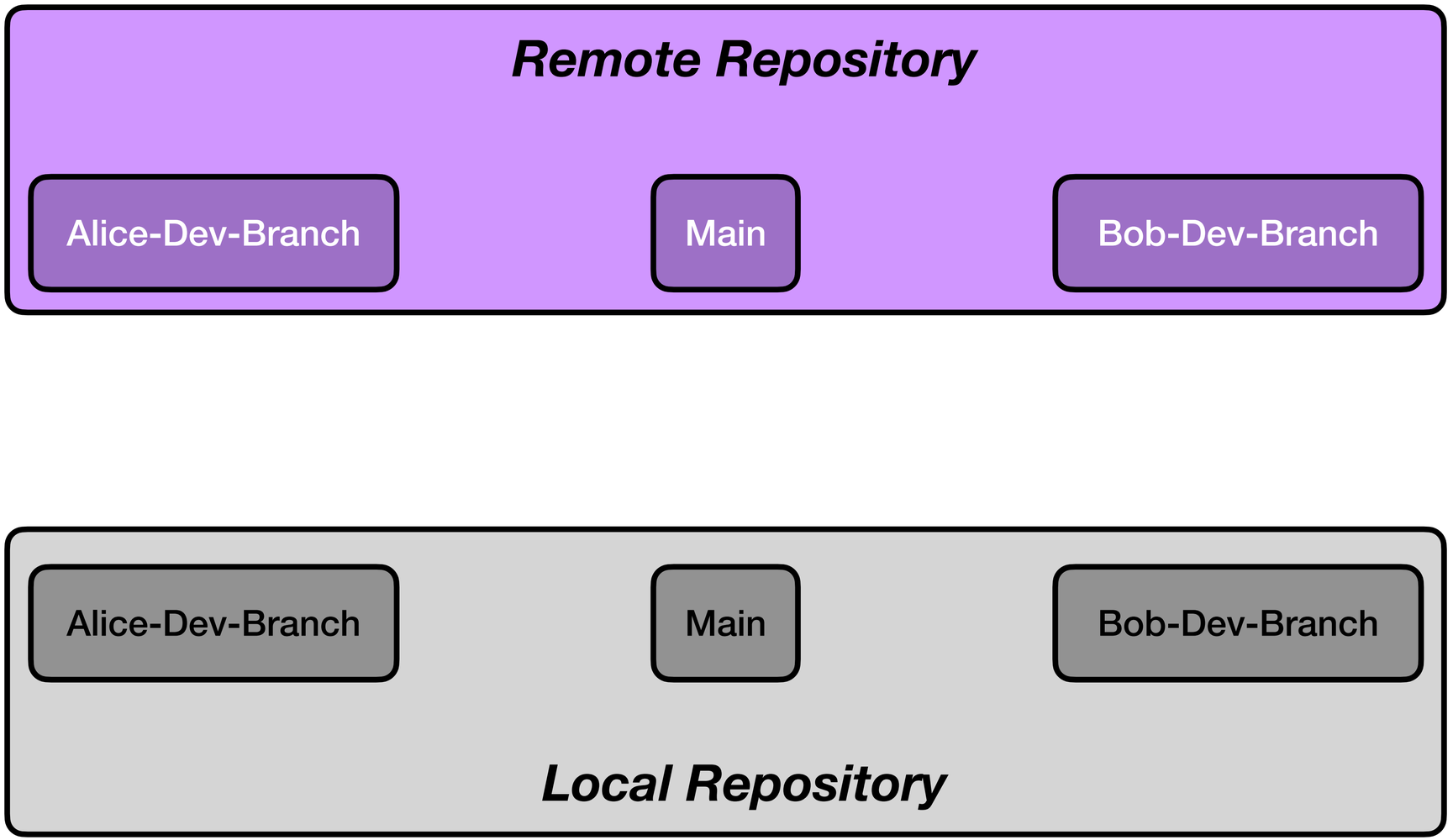
If you break something, it’s isolated to the current branch…



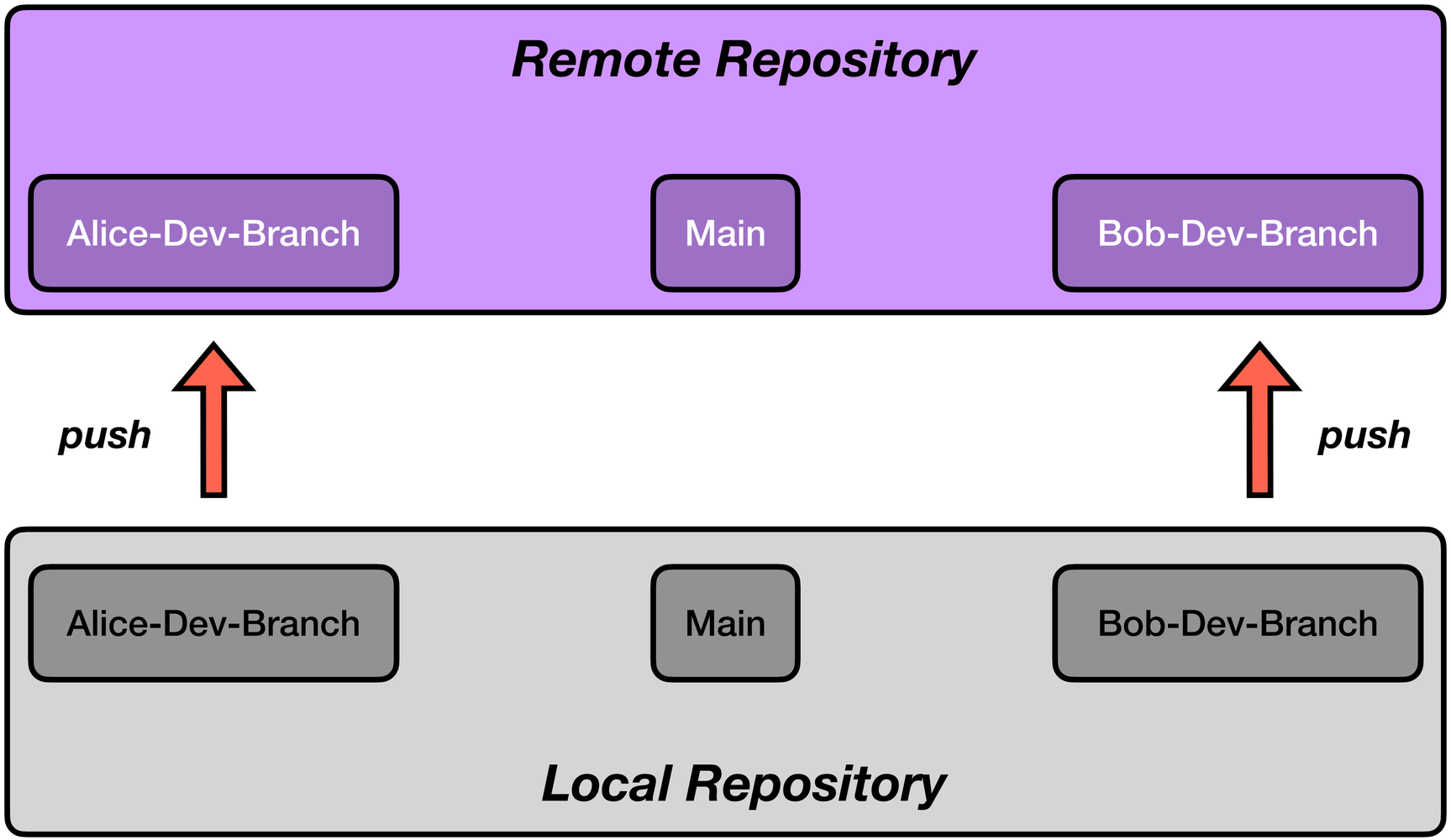
It’s fairly easy to resolve conflicts between branches!

The branch you are currently coding is called the ***working branch***

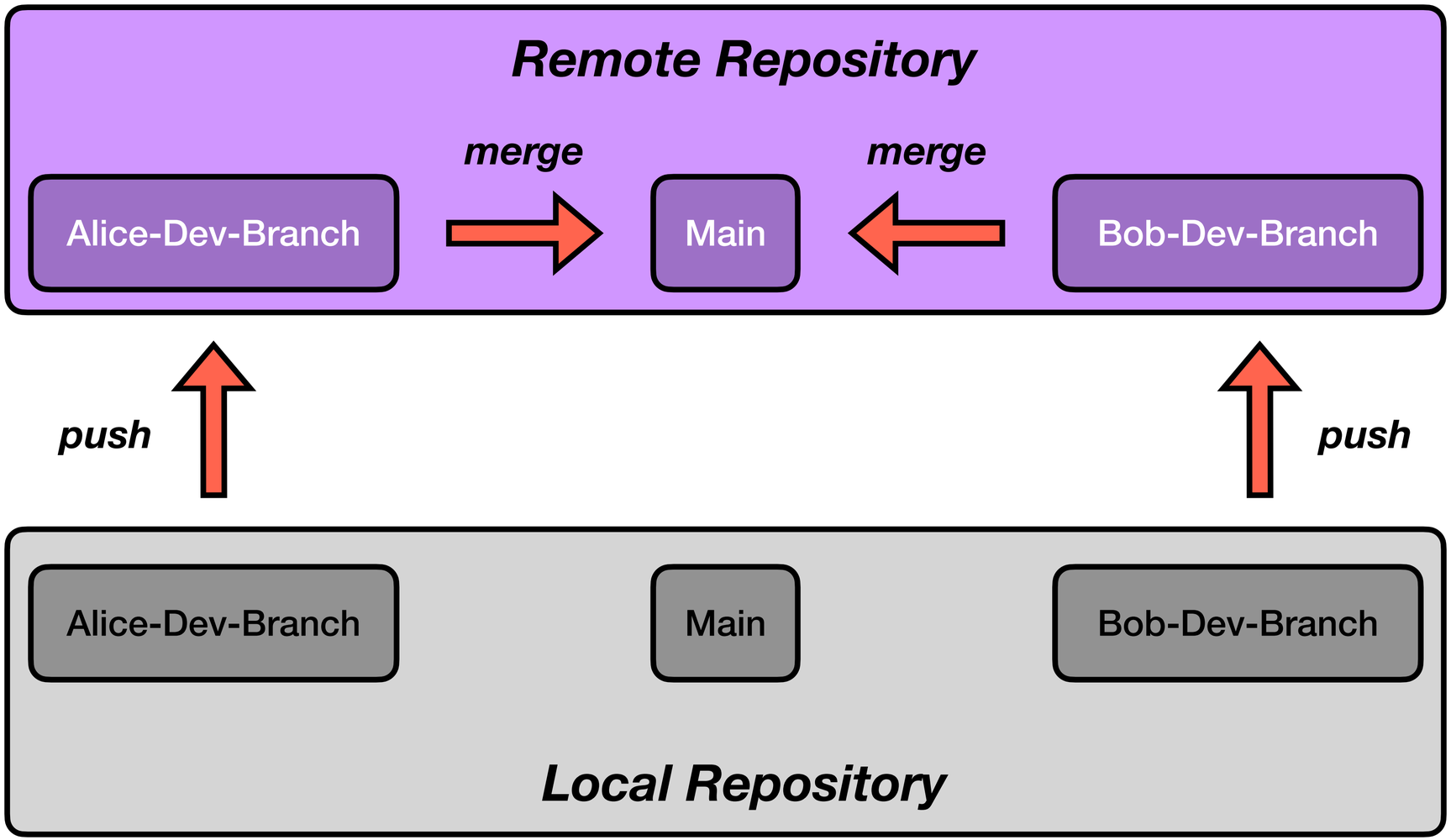
Code development should flow like a pot of boiling water…



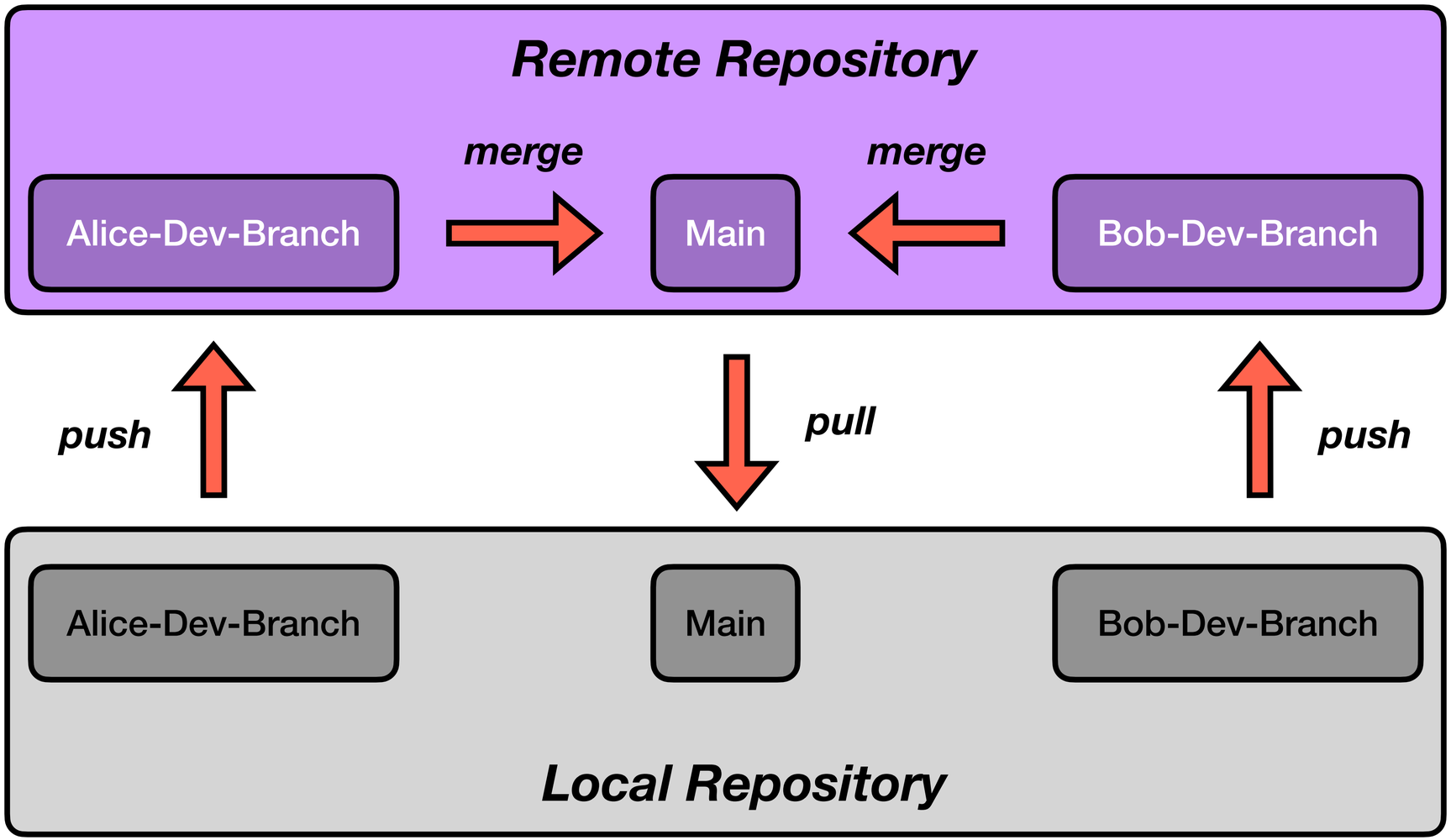
Code development should flow like a pot of boiling water…



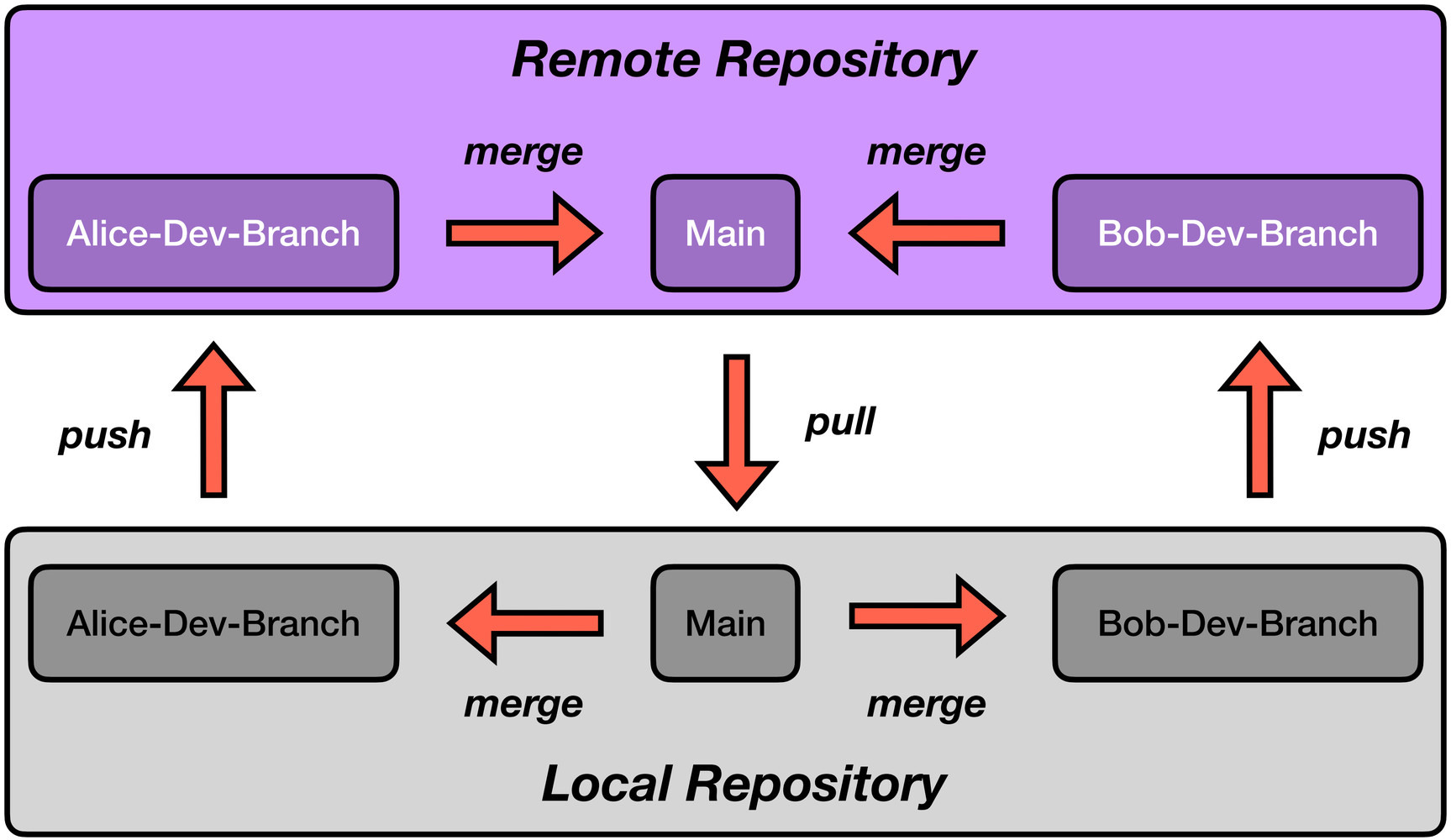
Code development should flow like a pot of boiling water…



Code development should flow like a pot of boiling water…



Code development should flow like a pot of boiling water…



Creating & Listing Existing Branches

```
> git branch branch-name
```

Creates a new branch on your local repository

```
> git branch
```

Lists all local branches (*current branch highlighted*)

```
(base) [redacted] practice_collaborative_coding % git branch autumn-dev
(base) [redacted] practice_collaborative_coding % git branch
  autumn-dev
* main
```

Creating & Listing Existing Branches

```
> git branch branch-name
```

Creates a new branch on your local repository

```
> git branch
```

Lists all local branches (*current branch highlighted*)

```
● (base)                                  practice_collaborative_coding % git branch autumn-dev
● (base)                                  practice_collaborative_coding % git branch
    autumn-dev
  * main
```

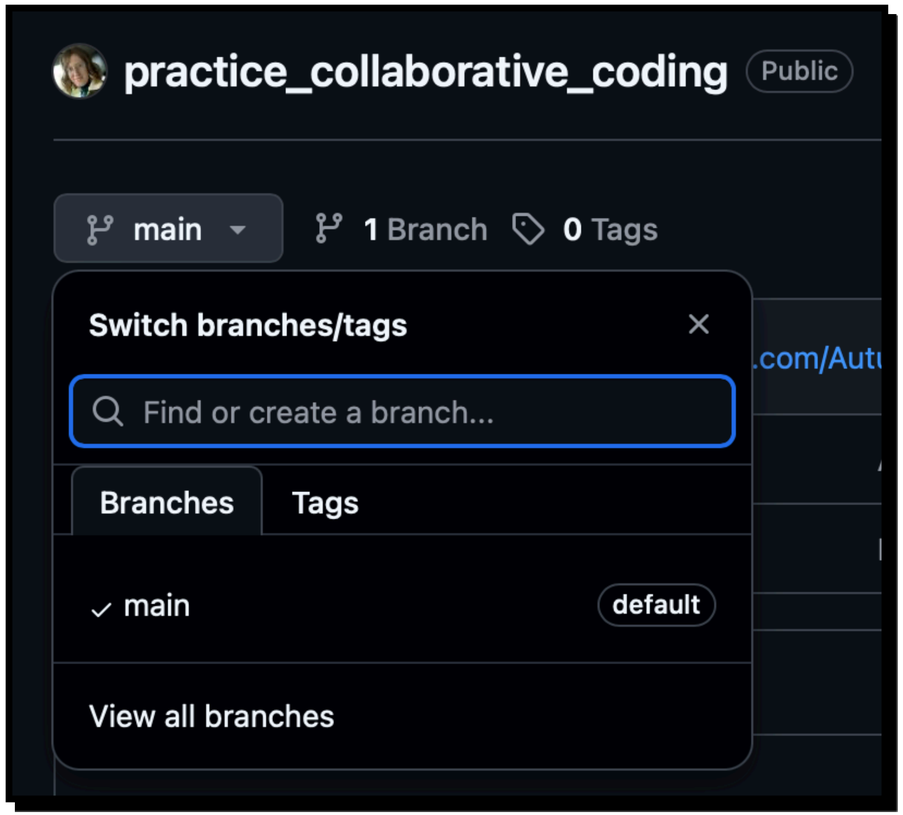
```
> git switch branch-name
```

Changes your current working branch

```
● (base)                                  practice_collaborative_coding % git switch autumn-dev
  Switched to branch 'autumn-dev'
● (base)                                  practice_collaborative_coding % git branch
  * autumn-dev
    main
```

Creating & Listing Existing Branches

*So far, we only created a **new branch** on our **local repository**. If we look at GitHub, we can see that the only listed branch is* `main` .



Creating & Listing Existing Branches

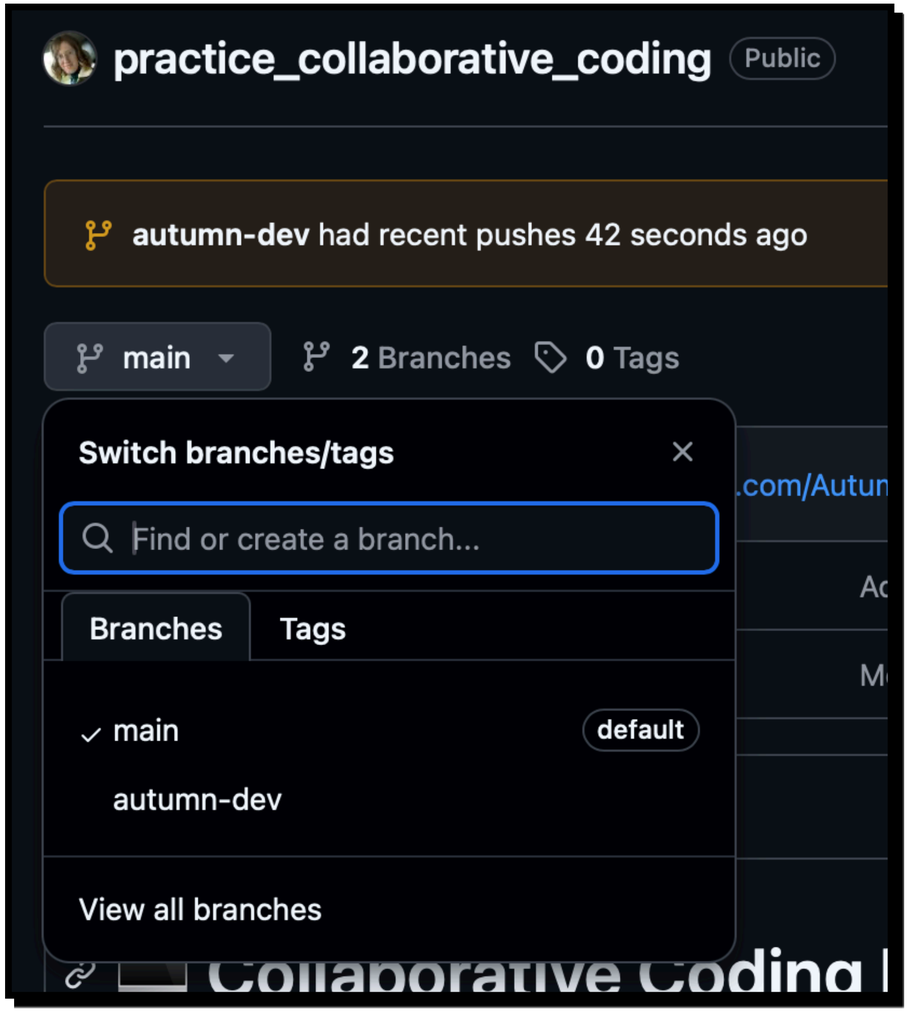
Let's add, commit, and push our change!

```
● (base) [redacted] practice_collaborative_coding % git add src/sample_code.py
● (base) [redacted] practice_collaborative_coding % git commit -m "Initial commit"
  [autumn-dev a209c7d] Initial commit
   1 file changed, 8 insertions(+)
   create mode 100644 src/sample_code.py
● (base) aust8150@UCB-G0PQL4Q917 practice_collaborative_coding % git push origin autumn-dev
```

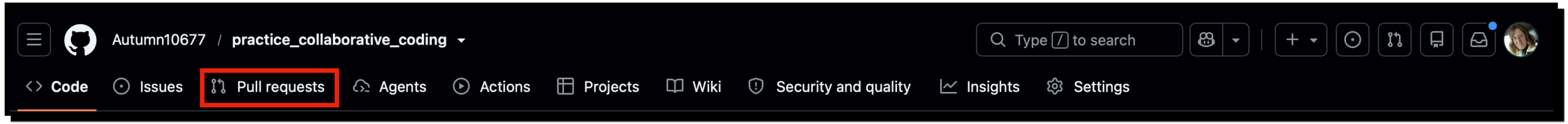
When pushing to a new branch, you get this message…

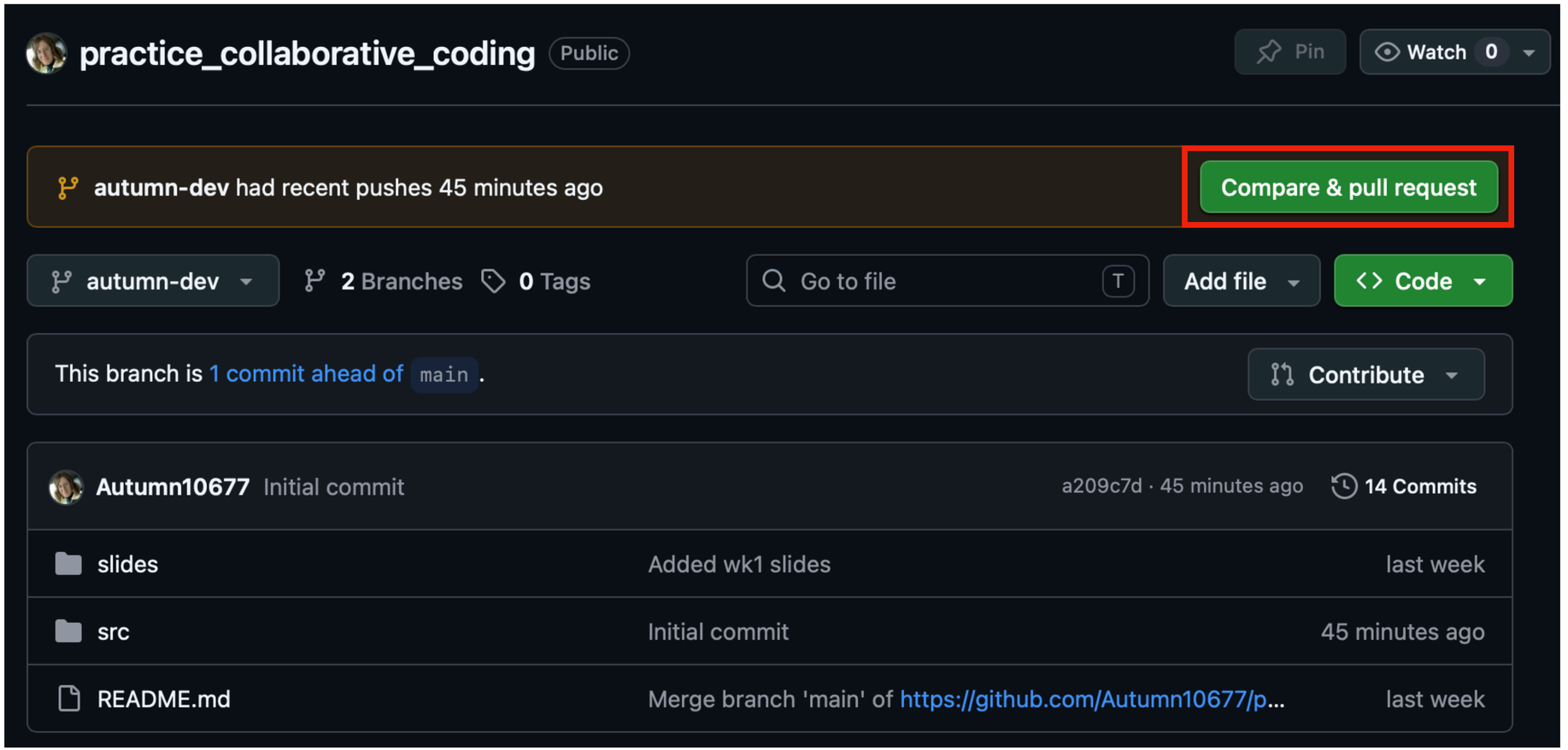
```
To https://github.com/Autumn10677/practice_collaborative_coding.git
 * [new branch]      autumn-dev -> autumn-dev
```

The GitHub repository has our branch!



Handling Merge Requests on GitHub



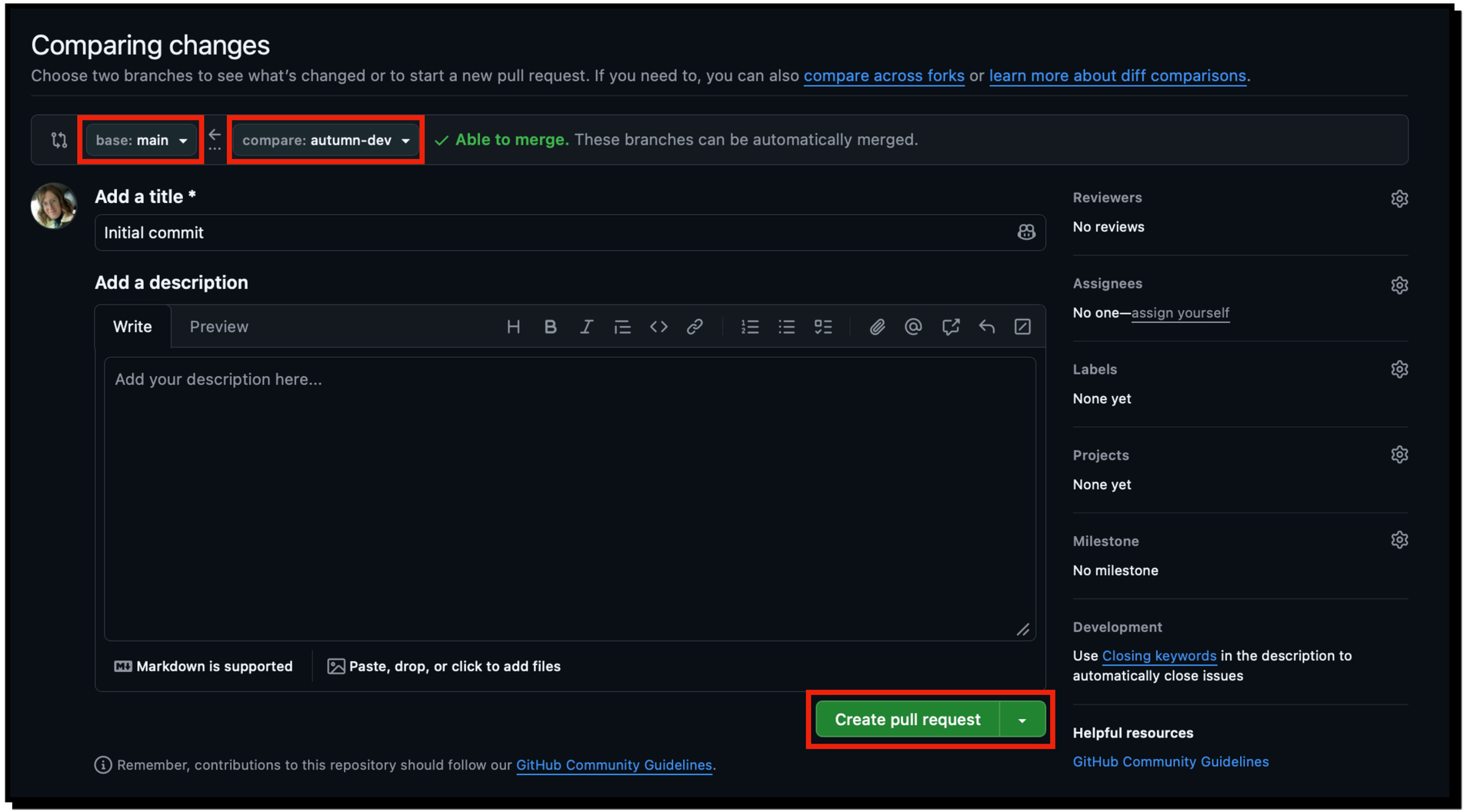


You can start a pull request using either of these options.

The "***Compare & pull request***" button usually only shows up for recent changes.

Handling Merge Requests on GitHub

Check the target / source branches, then create a pull request!



Most of the options on the side here are not important for now. These can help you organize merge requests as a project grows in size.

Handling Merge Requests on GitHub

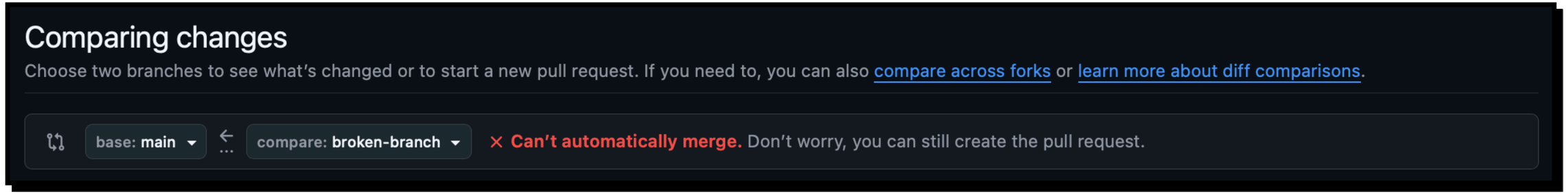
Don’t forget to update your local repository!

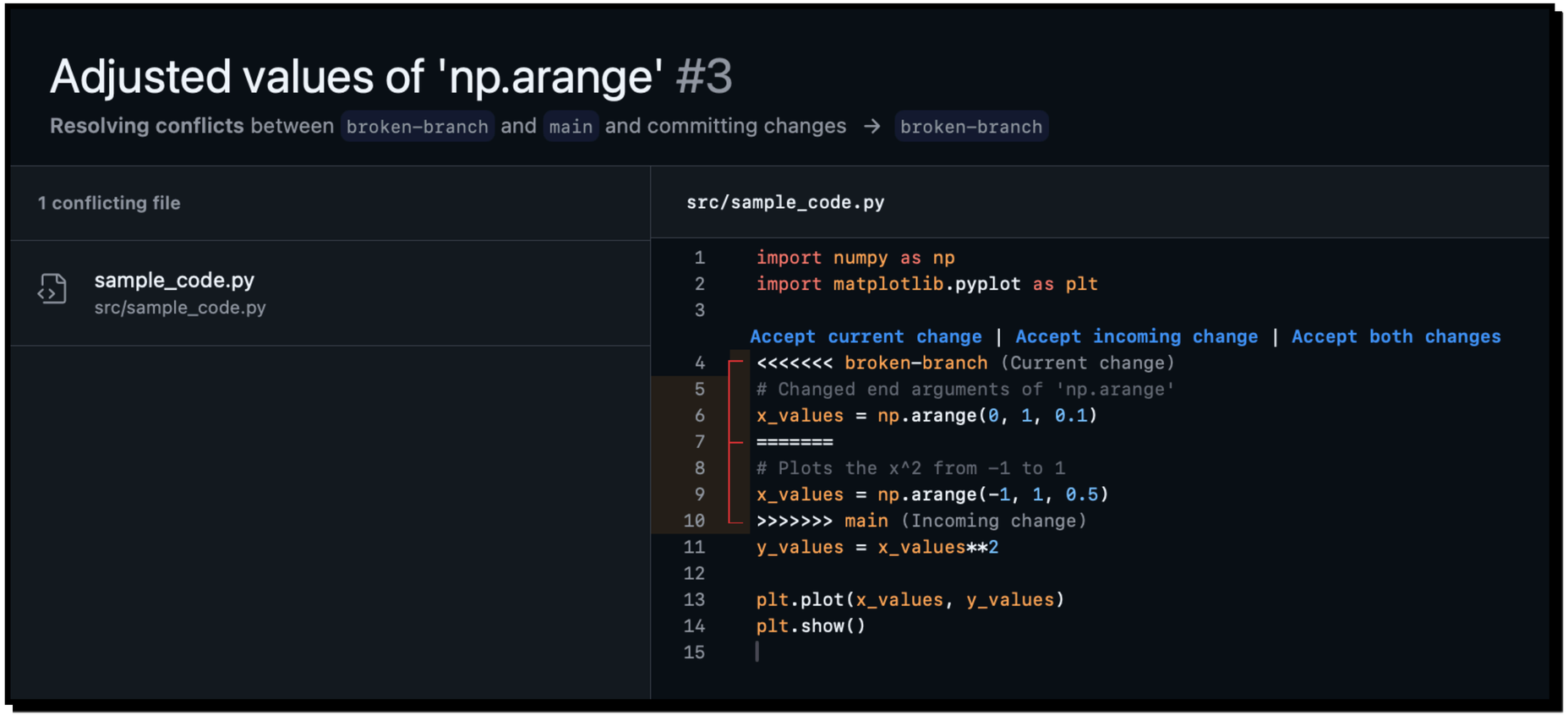
```
(base) [redacted]practice_collaborative_coding % git pull origin main
From https://github.com/Autumn10677/practice_collaborative_coding
 * branch            main       -> FETCH_HEAD
Updating e90ea0c..fd4390c
Fast-forward
 src/sample_code.py | 8 ++++++++
 1 file changed, 8 insertions(+)
 create mode 100644 src/sample_code.py
(base) [redacted]practice_collaborative_coding % git switch autumn-dev
Switched to branch 'autumn-dev'
(base) [redacted]practice_collaborative_coding % git merge main
```

As long as no conflicts were detected, this should be relatively simple.

What if there is a conflict?

Merge Conflicts



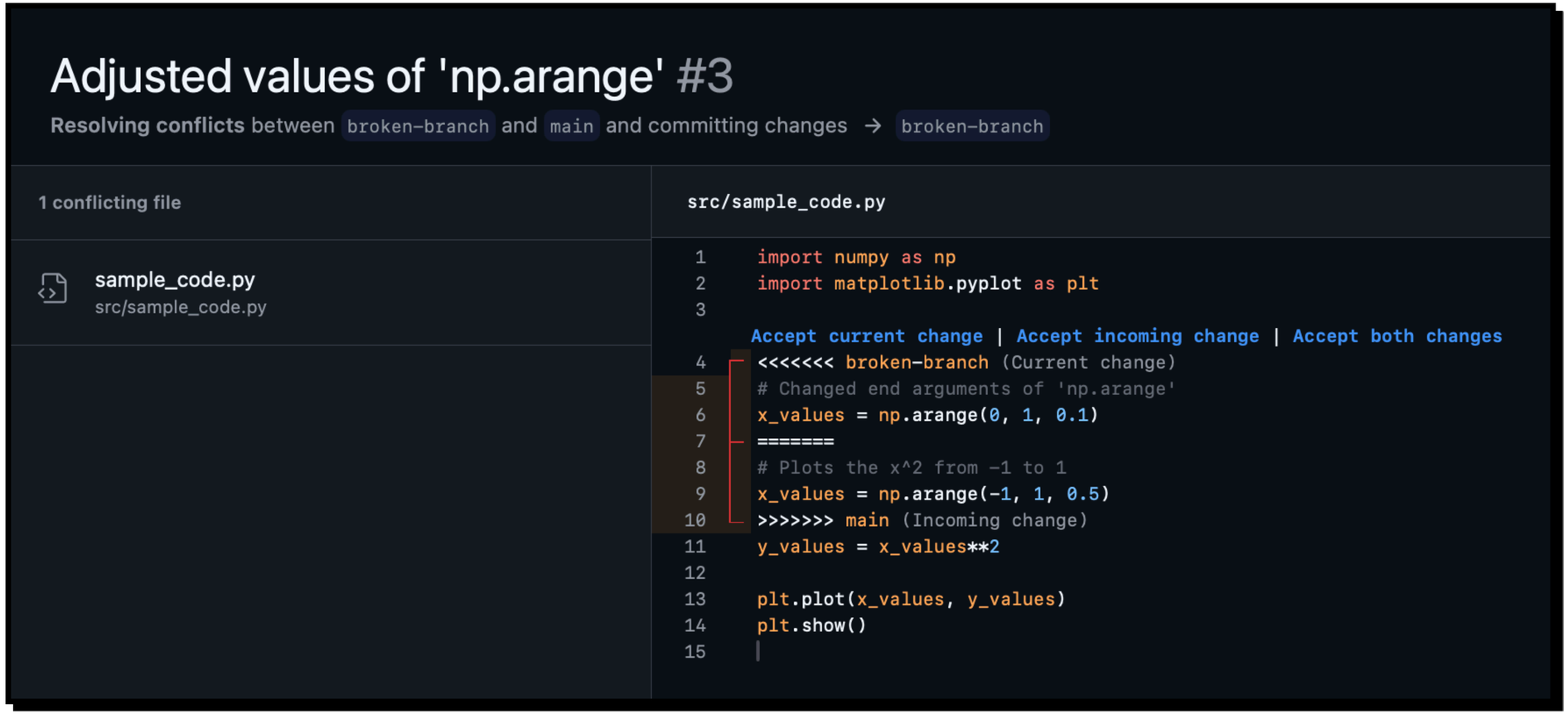


If conflicts are detected between your branch and main, you will need to tell GitHub or VS Code which changes to keep.

This is an example of a relatively minor conflict between two branches.

Merge Conflicts

There is no good general rule here, what you choose will depend a lot on your project!



Accept current change

```
# Changed end arguments of 'np.arange'
x_values = np.arange(0, 1, 0.1)
y_values = x_values**2
```

Accept incoming change

```
# Plots the x^2 from -1 to 1
x_values = np.arange(-1, 1, 0.5)
y_values = x_values**2
```

Accept both changes

```
# Changed end arguments of 'np.arange'
x_values = np.arange(0, 1, 0.1)
# Plots the x^2 from -1 to 1
x_values = np.arange(-1, 1, 0.5)
y_values = x_values**2
```

How *should* code be written?

How *should* code be written?

However you want! But, here are some suggestions…

Introduction to Best Practices

<u>A Non-Exhaustive List of Suggestions:</u>

- Make sure code fails fast
- Avoid “magic numbers”
- Don’t write repetitive code
- Clear, concise comments where needed
- Write single-purpose functions
- Return results, don’t print them
- Handle errors internally and control behavior
- Use descriptive variable / function names
- Follow a style guide

*All of these are meant to make code easy to **<u>test</u>** & **<u>read</u>**!*

Introduction to Best Practices

<u>A Non-Exhaustive List of Suggestions:</u>

- Make sure code fails fast
- Avoid “magic numbers”
- Don’t write repetitive code
- Clear, concise comments where needed
- Write single-purpose functions
- Return results, don’t print them
- Handle errors internally and control behavior
- Use descriptive variable / function names
- Follow a style guide

*All of these are meant to make code easy to **<u>test</u>** & **<u>read</u>**!*

*Let’s start with **style guides**!*

flake8: A Simple Checker for the PEP-8 Style Guide

This code would run, but we can improve readability!

sample_code.py > ...

```
1    import numpy as np
2    import matplotlib.pyplot as plt
3
4    x_values = np.arange(0,10,1)
5    y_values = x_values ** 2
```

flake8: A Simple Checker for the PEP-8 Style Guide

This code would run, but we can improve readability!

sample_code.py > ...

```
1    import numpy as np
2    import matplotlib.pyplot as plt
3
4    x_values = np.arange(0,10,1)
5    y_values = x_values ** 2
```

*Using **flake8**, we can get an easy-to-read list of style guide mistakes*

```
(base) [redacted] % flake8 sample_code.py
sample_code.py:2:1: F401 'matplotlib.pyplot as plt' imported but unused
sample_code.py:4:23: E231 missing whitespace after ','
sample_code.py:4:26: E231 missing whitespace after ','
sample_code.py:5:25: W292 no newline at end of file
```

flake8: A Simple Checker for the PEP-8 Style Guide

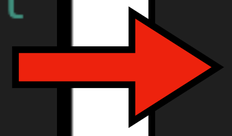
```
(base)                                                    % flake8 sample_code.py
sample_code.py:2:1: F401 'matplotlib.pyplot as plt' imported but unused
sample_code.py:4:23: E231 missing whitespace after ','
sample_code.py:4:26: E231 missing whitespace after ','
sample_code.py:5:25: W292 no newline at end of file
```

Each entry has the following format…

file_name *:* *Line #* *:* *Character #* *:* ***Rule ID*** */ Explanation*

sample_code.py

```
1   import numpy as np
2   import matplotlib.pyplot as plt
3
4   x_values = np.arange(0,10,1)
5   y_values = x_values ** 2
```



sample_code.py

```
1   import numpy as np
2
3   x_values = np.arange(0, 10, 1)
4   y_values = x_values ** 2
5
```

Other Style Guides

Black — Automatically fixes files for you!

```
(base)                                                % black sample_code.py
reformatted sample_code.py

All done! ✨ 🍰 ✨
1 file reformatted.
```

pylint — Conforms to PEP-8

pycodestyle — Also uses PEP-8

Ruff — Never used this one, but seems popular!

A Handy Cheat Sheet

`git…`

`…clone`	First-time **download and setup** of local a repository
`…fetch`	Informs the **local repository** about **remote branch names**
`…add`	**Stages** a changed file for saving
`…commit`	Create a **new save state** with all new changes
`…push`	**Update remote** repository **using local** changes
`…pull`	**Update local** repository **using remote** changes
`…branch`	**List** all **existing branches** on your **local** repository
`…branch xxx`	**Create** a **new branch** on your **local** repository
`…merge xxx`	**Merge** branch “xxx” into your **working branch**

Next Time…

Unit Testing & GitHub Actions